



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology,

Faculty of Computer Science and Engineering

CS101 – Compiler Construction Lab

Why Do We Need Code Optimization?

- To improve performance (faster execution).
- To reduce memory usage.
- To generate efficient machine code.
- To reduce power consumption (important in embedded systems).
- To enhance program scalability and maintainability.

What is Code Optimization?

- Code Optimization is the process of improving intermediate code or source code so that the resulting executable code runs more efficiently—either by making it run faster, take up less space, or both—without changing the program's output.

Types of Code Optimization:

1. **Machine-Independent Optimization:** These optimizations are not dependent on the hardware. They are performed on the intermediate code before generating machine code.

Techniques:

Constant Folding: Evaluate constant expressions at compile time.

```
// Before optimization
```

```
int a = 5 * 10;
```

```
// After optimization
```

```
int a = 50;
```

2. **Common Subexpression Elimination:** Avoid recomputing expressions already computed.

```
// Before
```

```
int a = (x * y) + z;
```

```
int b = (x * y) - z;
```

```
// After
```

```
int temp = x * y;
```

```
int a = temp + z;
```

```
int b = temp - z;
```

3. **Dead Code Elimination:** Remove code that doesn't affect the program's output.

```
// Before
```

```
int a = 10;
```

```
a = 20;
```

```
printf("%d", a);
```

```
// After
```

```
int a = 20;
```

```
printf("%d", a);
```

4. **Strength Reduction:** Replace expensive operations with cheaper ones.

```
// Before
```

```
int a = b * 8;
```

```
// After
```

```
int a = b << 3; // Multiplying by 8 using left shift
```

Machine-Dependent Optimization: These depend on the target architecture and are applied during or after code generation.

Techniques:

1. **Instruction Scheduling:** Rearrange instructions to avoid pipeline stalls on CPUs.

```
// Suppose two instructions use the same register in a row
// We schedule independent instructions in between to avoid delay

// Before
int a = b + c;
int d = a * 2;
int e = d - 1;

// After
int a = b + c;
int x = p + q; // Independent, can be scheduled in between
int d = a * 2;
int e = d - 1;
```

2. **Register Allocation:** Use CPU registers effectively instead of memory.

```
// Before
int a, b, c;
a = b + c; // values loaded from memory

// After
// a, b, c stored in registers → faster execution
```

3. Peephole Optimization: Optimize small sets of instructions (window/peephole) at a time.

```
// Before
```

```
x = x + 0;
```

```
// After
```

```
// Remove because adding 0 has no effect
```

Example:

Before Optimization:

```
#include <stdio.h>
int main() {
    int x = 5 * 10;
    int y = (x * 2) + (x * 2);
    int z = y + 0;
    printf("%d\n", z);
    return 0;
}
```

After Optimization:

```
#include <stdio.h>
int main() {
    int x = 50; // constant folding
    int temp = x * 2; // common subexpression elimination
    int z = temp + temp; // simplified expression
    printf("%d\n", z); // dead code removed
    return 0;
}
```

Code Files

Flex File (.l): “Lab08_Example.l”

```
%{
#include "Lab08_Example.tab.h"
#include <string.h>
%}

%%

[0-9]+      { yylval.str = strdup(yytext); return NUM; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.str = strdup(yytext); return ID; }

"="  { return ASSIGN; }
"+"  { return PLUS; }
"*"  { return MUL; }
";"  { return SEMICOLON; }
"("  { return LPAREN; }
")"  { return RPAREN; }

[ \t\n]+ ; // ignore whitespace

. { return yytext[0]; }
%%

int yywrap() {
    return 1;
}
```

Bison File (.y): “Lab08_Example.y”

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* Function declarations */
int yylex();
int yyerror(char *msg);

/* Temporary variable generator (t1, t2, ...) */
char tempVar[10];
int tempCount = 1;

/* Variables used for Common Subexpression Elimination (CSE) */
```

```

char lastExpr[50] = "";
char lastTemp[10] = "";

/* Function to generate Three Address Code (TAC)
Example: t1 = a * b */
void gen(char* target, char* src1, char* op, char* src2) {
    printf("%s = %s %s %s\n", target, src1, op, src2);
}

/* Function to handle assignment statements
Example: x = t1 */
void genAssign(char* id, char* val) {
    printf("%s = %s\n", id, val);
}

/* Define the type of semantic values */
%union {
    char* str;
}

/* Token definitions */
%token <str> ID NUM
%token ASSIGN PLUS MUL SEMICOLON LPAREN RPAREN

/* Operator precedence to resolve ambiguity */
%left PLUS
%left MUL

/* Non-terminal types */
%type <str> expr term factor

/* Start symbol */
%start program

%%

/* -----
Program Rule
Allows multiple statements in input
----- */
program : program stmt
        | stmt
        ;

```

```

/* -----
Statement Rule
Handles assignment statements
Example: x = expression;
----- */
stmt : ID ASSIGN expr SEMICOLON
{
    genAssign($1, $3);
}
;

/* -----
Expression Rule
Handles addition operations
----- */
expr : expr PLUS term
{
    /* Generate a new temporary variable */
    sprintf(tempVar, "%d", tempCount++);

    /* Generate TAC for addition */
    gen(tempVar, $1, "+", $3);

    /* Return the temporary variable */
    $$ = strdup(tempVar);
}
| term
{
    $$ = $1;
}
;

/* -----
Term Rule
Handles multiplication operations
----- */
term : term MUL factor
{
    /* Constant Folding:
    If both operands are numeric constants,
    compute the result at compile time */
    if (isdigit($1[0]) && isdigit($3[0])) {
        int val = atoi($1) * atoi($3);

        char buf[10];

```

```

    sprintf(buf, "%d", val);

    $$ = strdup(buf);
}
else {
    /* Common Subexpression Elimination (CSE):
       Avoid recomputing identical expressions */
    char exprStr[50];
    sprintf(exprStr, "%s*%s", $1, $3);

    if (strcmp(exprStr, lastExpr) == 0) {
        /* Reuse previously computed result */
        $$ = strdup(lastTemp);
    } else {
        /* Create new temporary variable */
        sprintf(tempVar, "t%d", tempCount++);

        /* Generate TAC */
        gen(tempVar, $1, "*", $3);

        /* Store expression for reuse */
        strcpy(lastExpr, exprStr);
        strcpy(lastTemp, tempVar);

        $$ = strdup(tempVar);
    }
}
}
| factor
{
    $$ = $1;
}
;

/* -----
Factor Rule
Handles identifiers, numbers, and parentheses
----- */
factor : ID
{
    $$ = $1; /* Identifier */
}
| NUM
{
    $$ = $1; /* Numeric constant */
}

```



```

}
| LPAREN expr RPAREN
{
    $$ = $2; /* Expression inside parentheses */
}
;

%%

/* -----
Main Function
----- */
int main() {
    yyparse(); /* Start parsing */
    return 0;
}

/* -----
Error Handling Function
----- */
int yyerror(char* msg) {
    printf("Syntax Error: %s\n", msg);
    return 0;
}

```

How to Compile and Run the Program

```
bison -d Lab08_Example.y
```

```
flex Lab08_Example.l
```

```
gcc lex.yy.c Lab08_Example.tab.c -o output
```

```
.\output
```

Sample Inputs:

1. $x = 5 * 10;$

2. $z = (x * 2) + (x * 2);$

3. $z = (2 * 2) + (5 * 2);$

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'CS424L' with files: 'Lab08_Example.l', 'Lab08_Example.tab.c', 'Lab08_Example.tab.h', 'Lab08_Example.y', 'lex.yy.c', and 'output.exe'. The main editor displays the contents of 'Lab08_Example.l', which is a Bison grammar file. It defines tokens for assignment, plus, multiply, semicolon, left parenthesis, right parenthesis, and whitespace. The grammar rules are simple: 'ASSIGN' returns 'ASSIGN;', 'PLUS' returns 'PLUS;', 'MUL' returns 'MUL;', 'SEMICOLON' returns 'SEMICOLON;', 'LPAREN' returns 'LPAREN;', and 'RPAREN' returns 'RPAREN;'. There is also a rule for whitespace that ignores it. The file ends with a call to 'yywrap()' which returns 1. The bottom pane shows the 'TERMINAL' output. It shows the commands: 'bison -d Lab08_Example.y', 'flex Lab08_Example.l', and 'gcc lex.yy.c Lab08_Example.tab.c -o output'. The output shows the execution of the program, which prints the value of 'a' (4) and then returns 0.

```
File Edit Selection View Go Run Terminal Help
CS424L [Administrator]

EXPLORER
CS424L
  Lab08_Example.l
  Lab08_Example.tab.c
  Lab08_Example.tab.h
  Lab08_Example.y
  lex.yy.c
  output.exe

Lab08_Example.l
11 "=" { return ASSIGN; }
12 "+" { return PLUS; }
13 "*" { return MUL; }
14 ";" { return SEMICOLON; }
15 "(" { return LPAREN; }
16 ")" { return RPAREN; }
17
18 [\t\n]+; // ignore whitespace
19
20 { return yytext[0]; }
21
22 %%
23
24 int yywrap() {
25     return 1;
26 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Administrator\Desktop\CS424L> bison -d Lab08_Example.y
PS C:\Users\Administrator\Desktop\CS424L> flex Lab08_Example.l
PS C:\Users\Administrator\Desktop\CS424L> gcc lex.yy.c Lab08_Example.tab.c -o output
c:/mingw/bin/../../lib/gcc/mingw32/6.3.0/../.././../mingw32/bin/ld.exe: cannot open output file output.exe: Permis-
sion denied
collect2.exe: error: ld returned 1 exit status
PS C:\Users\Administrator\Desktop\CS424L> .\output
x = 5 * 10;
x = 50
z = (x * 2) + (x * 2);
t1 = x * 2
t2 = t1 + t1
z = t2
z = (2 * 2) + (5 * 2);
t3 = 4 + 10
z = t3
}
```

Task 01: Apply constant folding, common subexpression elimination, strength reduction, and dead code elimination to optimize the code.

```
int main() {

    int x = 4 * 5;

    int y = (x + 10) - (x + 10);

    int z = 10 * 8;

    int a = z / 2;

    int b = 100;

    b = 200; // Unused assignment

    printf("%d\n", a);

    return 0;

}
```

Task 02: Identify and optimize by applying peephole optimizations, such as:

- Removing redundant operations
- Replacing expensive operations with cheaper ones
- Reordering independent instructions (instruction scheduling)

```
int main() {  
    int a = 10;  
    int b = a + 0;  
    int c = b * 1;  
    int d = 5;  
    int e = d * 8;  
    int f = a + b;  
    int g = c + d;  
    printf("%d %d\n", f, g);  
    return 0;  
}
```